

Percorso completo per creare un gioco tattico simultaneo in Unreal Engine

Sintesi esecutiva

L'obiettivo del percorso è realizzare una **vertical slice giocabile** di un titolo tattico a turni simultanei, ispirato alla struttura di Atlas Reactor ma con identità, personaggi, ambientazione, nomi e interfaccia originali.

Il prototipo finale, chiamato provvisoriamente **Reactor Tactics**, comprenderà:

- un'arena tattica in visuale isometrica;
- due squadre da due unità;
- una modalità di pianificazione simultanea;
- una fase di risoluzione composta da **Prep**, **Dash**, **Blast** e **Move**;
- movimento su griglia, targeting, danni, scudi, cooldown ed energia;
- un avversario controllato dall'intelligenza artificiale;
- interfaccia, log di combattimento, semplici animazioni ed effetti;
- test automatici;
- build Windows distribuibile;
- una pipeline Git e una prima configurazione CI.

Il riferimento meccanico deriva dalla documentazione storica ufficiale di Atlas Reactor: tutti i giocatori selezionavano contemporaneamente le azioni durante la Decision Mode e, dopo il lock-in, il gioco le risolveva nelle quattro fasi ordinate **Prep → Dash → Blast → Move**. Il gioco originale usava inoltre ruoli, energia per le ultimate, coperture, linea di vista e combattimenti 4v4. ¹

Il corso non proverà a ricostruire subito l'intero gioco originale. Multiplayer online, matchmaking, ranking, sei o più eroi, replay e sistemi competitivi trasformerebbero un progetto didattico in uno scope monster. La prima versione sarà quindi **2v2 contro bot**, giocabile offline, con tre abilità per unità e una sola arena.

La strategia tecnica consigliata è:

Area	Tecnologia principale
Regole dei turni	C++
Strutture dati condivise	C++ esposto ai Blueprint
Matematica della griglia	C++
Prototipazione delle abilità	Blueprint
Camera e input	Blueprint con classi base C++
Interfaccia	UMG e Blueprint

Area	Tecnologia principale
Animazioni, audio e VFX	Blueprint
Configurazione delle unità	Data Asset e Data Table
Test della logica	C++ Automation Tests
Test end-to-end	Functional Test Blueprint
Build e packaging	Unreal Automation Tool

Epic considera Blueprint e C++ strumenti complementari: Blueprint è ottimo per iterazione, presentazione e logica ad alto livello, mentre C++ offre maggiore accesso all'engine, controllo dei dati, riuso, testabilità e prestazioni per sistemi intensivi. ²

Per questo progetto consiglio un rapporto iniziale di circa:

- **settanta per cento Blueprint;**
- **trenta per cento C++.**

La percentuale di C++ crescerà naturalmente man mano che imparerai il motore. Non è necessario "imparare tutto il C++" prima di iniziare: imparerai soltanto le parti utili al gioco, collegandole ai concetti che conosci già da C#.

La versione di riferimento è **Unreal Engine 5.8**, preferibilmente l'ultimo hotfix 5.8 disponibile nel Launcher. UE 5.8 è stata pubblicata il 23 giugno 2026 ed Epic la descrive come l'ultima major release UE5 attualmente pianificata, pur continuando a fornire correzioni e riservandosi la possibilità di ulteriori release. ³

Il percorso comprende **ventisei lezioni**, per circa **centoventi-centosessanta ore complessive**, inclusi esercizi e debugging. Con un ritmo di otto-dodici ore alla settimana, la vertical slice è realisticamente raggiungibile in tre-quattro mesi.

Assunzioni, obiettivi e architettura didattica

Assunzioni di progetto

Elemento	Assunzione
Budget iniziale	Da zero a cento euro
Team	Una persona
Ritmo	Otto-dodici ore settimanali
Sistema principale	Windows 11 x64
Piattaforma di destinazione	PC Windows
Input	Mouse e tastiera
Modalità iniziale	Offline contro bot
Dimensione delle squadre	Due contro due

Elemento	Assunzione
Arena	Una mappa graybox
Personaggi	Due archetipi iniziali
Multiplayer online	Post-tutorial
Grafica	Placeholder e asset gratuiti
Distribuzione	Build privata o demo non commerciale
Proprietà intellettuale	Ambientazione e contenuti originali

Il nome Atlas Reactor viene usato esclusivamente come riferimento per analizzare una struttura di gameplay. Per una pubblicazione reale dovranno essere originali almeno personaggi, nomi, lore, abilità, mappe, branding, UI, icone, audio e asset promozionali.

Obiettivi di apprendimento

Al termine del percorso saprai:

1. installare e configurare Unreal Engine, Visual Studio, Git e Git LFS;
2. orientarti nell'Editor e nel Gameplay Framework;
3. capire le principali differenze tra C# e Unreal C++;
4. creare classi `Actor`, `Pawn`, `PlayerController`, `GameMode` e `GameState`;
5. usare `UCLASS`, `USTRUCT`, `UENUM`, `UPROPERTY` e `UFUNCTION`;
6. collegare classi C++ e Blueprint;
7. implementare input, selezione e camera tattica;
8. costruire una griglia logica indipendente dalla grafica;
9. creare un sistema di turni simultanei deterministico;
10. rappresentare azioni pianificate come dati;
11. creare abilità data-driven;
12. implementare movimento, targeting, danni, scudi e status;
13. creare UI con UMG;
14. progettare un bot semplice ma leggibile;
15. usare log, breakpoint, debugger e Unreal Insights;
16. scrivere test automatici;
17. creare build Development e Shipping;
18. configurare un repository e una pipeline CI basilare.

Definizione della vertical slice

La vertical slice è completa quando soddisfa questa checklist:

Area	Criterio di completamento
Avvio	Il gioco apre direttamente l'arena
Squadre	Sono presenti due unità alleate e due nemiche
Selezione	Il giocatore può selezionare le proprie unità
Pianificazione	Ogni unità può scegliere abilità, bersaglio e movimento

Area	Criterio di completamento
Lock-in	Il giocatore può confermare il turno
Bot	Le unità nemiche pianificano automaticamente
Risoluzione	Le azioni seguono le quattro fasi
Combattimento	Danni, scudi e morte funzionano
Vittoria	La partita termina quando una squadra viene eliminata
UI	Salute, energia, abilità, timer e fase sono visibili
Testing	Esistono test per griglia, danno e ordine delle fasi
Build	La build Windows si avvia su un PC senza Editor

Modello mentale C# → Unreal

Concetto noto in C#	Equivalente o concetto vicino in Unreal
<code>class</code>	Classe C++, spesso annotata con <code>UCLASS()</code>
Proprietà serializzata	<code>UPROPERTY()</code>
Metodo esposto all'Editor	<code>UFUNCTION(BlueprintCallable)</code>
<code>List<T></code>	<code>TArray<T></code>
<code>Dictionary<TKey, TValue></code>	<code>TMap<TKey, TValue></code>
<code>HashSet<T></code>	<code>TSet<T></code>
<code>event</code>	Delegate Unreal
<code>null</code>	<code>nullptr</code>
<code>ScriptableObject</code> Unity	<code>UDataAsset</code>
Prefab Unity	Blueprint Class
<code>GameObject</code>	Nessun equivalente unico; spesso <code>AActor</code>
Component	<code>UActorComponent</code> o <code>USceneComponent</code>
Coroutine	Timer, state machine, latent action o async task
Garbage collector .NET	Garbage collection di <code>UObject</code> , con regole proprie
Reflection C#	Unreal Header Tool e sistema di reflection
Namespace	Namespace C++, ma le classi Unreal seguono anche prefissi
DLL/progetto	Modulo Unreal definito da <code>.Build.cs</code>

Epic ha una guida specifica per sviluppatori provenienti da Unity che evidenzia come Unreal introduca un Gameplay Framework più strutturato, con ruoli distinti per GameMode, GameState, PlayerController, Pawn e PlayerState. ⁴

Regola architetturale centrale

La risoluzione del turno deve dipendere da:

```
Stato iniziale
+ Azioni confermate
+ Regole
+ Eventuale seed casuale
= Stato finale
```

Non deve dipendere dal frame rate, dalla durata delle animazioni o da `DeltaSeconds`.

Questa separazione rende il sistema:

- più facile da testare;
- più adatto al multiplayer;
- riproducibile;
- meno vulnerabile a bug legati alle animazioni;
- utilizzabile in futuro per replay e spettatori.

Configurazione completa dell'ambiente

Hardware consigliato

La documentazione corrente di Unreal raccomanda, per lo sviluppo Windows, un processore quad-core moderno, 32 GB di RAM, una GPU DirectX 12 con almeno 8 GB di memoria video e driver aggiornati.

5

Componente	Minimo pratico per il corso	Consigliato
CPU	Sei core moderni	Otto-dodici core
RAM	16 GB	32 GB
GPU	6 GB VRAM	8-12 GB VRAM
Disco libero	150 GB SSD	250 GB NVMe
Schermo	1920×1080	2560×1440
Sistema	Windows 10 22H2 x64	Windows 11 x64
Connessione	Necessaria per download	Fibra o connessione stabile

Con 16 GB di RAM il corso rimane fattibile, ma è opportuno chiudere browser pesanti, ridurre la qualità dell'Editor e limitare applicazioni aperte durante shader compilation e packaging.

Installazione di Unreal Engine

1. Crea o usa un account Epic Games.
2. Scarica [Epic Games Launcher](#).

3. Installa e apri il Launcher.
4. Entra nella sezione **Unreal Engine → Library**.
5. Premi il pulsante .
6. Seleziona Unreal Engine .
7. Premi **Options** prima dell'installazione.
8. Mantieni:
 9. Core Components;
 10. Starter Content;
 11. Templates and Feature Packs;
 12. supporto Windows.
13. Puoi disattivare temporaneamente:
 14. Android;
 15. iOS;
 16. Linux;
 17. simboli di debug dell'Editor, se lo spazio è limitato.
18. Completa l'installazione.
19. Nel menu della versione installata seleziona **Verify** se il primo avvio genera errori.

Risorse:

- [Download ufficiale di Unreal Engine](#)
- [Specifiche hardware e software](#)
- [Risorse Unreal in italiano](#)

Installazione di Visual Studio

Le integrazioni Microsoft permettono di visualizzare log Unreal, espandere macro, creare classi, individuare riferimenti Blueprint, eseguire test e fare debugging del C++ chiamato dai Blueprint. 

Puoi installare Visual Studio Community da PowerShell:

```
winget install --id Microsoft.VisualStudio.Community --exact
```

WinGet usa la forma generale `winget install <pacchetto>` e può richiedere elevazione amministrativa durante l'installazione. 

Al termine:

1. Apri **Visual Studio Installer**.
2. Premi **Modify** sulla versione installata.
3. Seleziona il workload **Game development with C++**.
4. Nel pannello dei componenti opzionali verifica:

5. Visual Studio Tools for Unreal Engine;
6. Visual Studio debugger tools for Unreal Engine Blueprints;
7. Unreal Engine Test Adapter;
8. MSVC C++ x64/x86 build tools;
9. Windows 10 o Windows 11 SDK;
10. C++ profiling tools.
11. Premi **Modify**.
12. Riavvia Windows dopo l'installazione.

Questi sono i componenti indicati dalla guida Microsoft ufficiale per lo sviluppo Unreal. 8

Dopo aver creato il progetto:

1. Apri il file `.sln`.
2. In Visual Studio seleziona **Project → Configure Tools for Unreal Engine**.
3. Verifica che l'integrazione del progetto sia verde.
4. Installa il plugin di integrazione nel progetto qualora il Test Adapter lo richieda. 9

Installazione di Git e Git LFS

Apri PowerShell come amministratore:

```
winget install --id Git.Git --exact
winget install --id GitHub.GitLFS --exact
git lfs install
```

Configura l'identità:

```
git config --global user.name "Aaron"
git config --global user.email "tuo-indirizzo@example.com"
git config --global init.defaultBranch main
git config --global core.autocrlf true
```

Git LFS registra i pattern tracciati nel file `.gitattributes`; questo file deve essere incluso nel repository insieme agli asset. 10

Creazione del progetto

1. Avvia Unreal Engine.
2. Seleziona **Games**.
3. Scegli **Blank**.
4. Seleziona:
5. `C++`;
6. `Desktop`;
7. qualità `Maximum`;

8. Ray Tracing disattivato;
9. Starter Content attivo.
10. Nome: `ReactorTactics`.
11. Percorso consigliato:

D:\Dev\ReactorTactics

1. Evita cartelle sincronizzate automaticamente come OneDrive.
2. Evita spazi, lettere accentate e caratteri speciali nel percorso.
3. Premi **Create**.
4. Attendi la generazione della soluzione e la prima compilazione.

Test immediato della toolchain

Nell'Editor:

1. Seleziona **Tools → New C++ Class**.
2. Scegli `Actor`.
3. Nome: `ToolchainTestActor`.
4. Premi **Create Class**.
5. Attendi l'apertura di Visual Studio.
6. Compila con `Ctrl+Shift+B`.
7. Torna nell'Editor.
8. Cerca `ToolchainTestActor` nel Content Browser o nella finestra di creazione Blueprint.

Il test è riuscito quando:

- Visual Studio compila senza errori;
- l'Editor resta aperto;
- la classe compare tra le classi disponibili;
- puoi creare un Blueprint figlio.

Dopo il test elimina la classe con cautela:

1. chiudi l'Editor;
2. elimina `.h` e `.cpp`;
3. rimuovi eventuali riferimenti;
4. ricompila il progetto;
5. riapri l'Editor.

Inizializzazione del repository

Dalla cartella del progetto:

```
cd D:\Dev\ReactorTactics
git init
git lfs install
git lfs track "*.uasset"
```



```
git lfs track "*.umap"  
git add .gitattributes
```

Crea `.gitignore`:

```
.vs/  
Binaries/  
DerivedDataCache/  
Intermediate/  
Saved/  
Script/  
*.VC.db  
*.opensdf  
*.sdf  
*.sln  
*.suo  
*.xcodeproj
```

Non ignorare:

```
Config/  
Content/  
Plugins/  
Source/  
ReactorTactics.uproject  
ReactorTactics.Target.cs  
ReactorTacticsEditor.Target.cs
```

Primo commit:

```
git add .  
git commit -m "Create Unreal Engine project"
```

Il template `.gitignore` ufficiale GitHub per Unreal può essere usato come base più completa. ¹¹

Configurazione dell'Editor

In **Edit** → **Editor Preferences**:

- abilita il salvataggio automatico, ma non considerarlo un sostituto di Git;
- imposta il salvataggio automatico ogni cinque-dieci minuti;
- abilita **Show Frame Rate and Memory**;
- abilita **Display UI Extension Points** solo quando necessario;
- usa Live Coding soltanto per modifiche locali ai metodi;
- chiudi l'Editor prima di modificare costruttori, proprietà riflesse, header o gerarchie di classi.

In **Edit** → **Project Settings**:

- Maps & Modes → imposta in seguito `L_Arena` come Game Default Map;
- Input → useremo Enhanced Input;
- Collision → aggiungeremo i canali `Grid` e `Unit`;
- Gameplay Tags → abiliteremo i tag nelle lezioni dedicate;
- Packaging → configura le mappe da includere.

Strategia tecnica e struttura del progetto

Confronto tra Blueprint, C++ e C#

Opzione	Vantaggi	Svantaggi	Uso consigliato
Blueprint	Visuale, rapido, integrato, ottimo per UI e prototipi	Graph grandi, merge difficili, meno adatto a loop pesanti	UI, camera, presentazione, abilità prototipo
C++	Supporto ufficiale completo, testabile, performante, diff testuale	Compilazione, macro Unreal, gestione header più complessa	Griglia, turni, dati, risoluzione, test
UnrealCLR	Sintassi C# familiare, .NET e NuGet	Plugin esterno, API e compatibilità dipendono dal progetto community	Esperimenti, non percorso principale
UnrealSharp	Binding generati, C# moderno, hot reload	Plugin community e ulteriore dipendenza produttiva	Side project dopo la vertical slice

Blueprint è un sistema completo di scripting visuale capace di definire classi orientate agli oggetti e richiamare funzionalità native dell'engine. Il costo maggiore compare soprattutto quando grandi quantità di nodi vengono eseguite continuamente; entrare in una funzione nativa da Blueprint non rende automaticamente lenta l'intera funzione. ¹²

UnrealCLR integra un host .NET 6 e consente l'uso di C# 10, caricamento di assembly, debugging e accesso a funzionalità gestite da Blueprint. Il suo README continua però a indicare requisiti e toolchain basati su .NET 6 e compatibilità da Unreal 4.25.4 in avanti, senza rappresentare il percorso ufficiale Epic. ¹³

UnrealSharp è un'alternativa community più recente: dichiara supporto a Unreal 5.6-5.8, .NET 10, binding generati e hot reload. Rimane comunque una dipendenza esterna, con supporto piattaforme e roadmap distinti dall'engine. ¹⁴

Decisione consigliata

Non usare UnrealCLR nel progetto principale.

La ragione non è che C# sia inadatto al gameplay, ma che questo corso ha anche l'obiettivo di insegnare Unreal. Affidarsi subito a un layer esterno introdurrebbe tre problemi contemporaneamente:

1. imparare Unreal;
2. imparare il plugin;
3. diagnosticare se un errore appartiene al gioco, all'engine o all'interoperabilità.

Il percorso principale userà:

```
Blueprint per imparare e iterare
+
C++ semplice per creare fondamenta solide
+
C# in seguito per script UAT e automazione
```

Unreal Automation Tool è scritto e ampliabile in C#, quindi le competenze .NET potranno essere riutilizzate in modo ufficialmente previsto per build, cooking, test e automazione. ¹⁵

Distribuzione delle responsabilità

Sistema	C++	Blueprint
Enum delle fasi	Sì	Lettura
Azione pianificata	Sì	Compilazione dei dati
Turn manager	Sì	Eventi visuali
Conversione griglia/mondo	Sì	Chiamata delle funzioni
Pathfinding iniziale	Sì	Visualizzazione percorso
Danno e scudi	Sì	Feedback visivo
Ability data	Base C++	Data Asset
Selezione unità	Base opzionale	Sì
Camera	No	Sì
Widget	No	Sì
Animazioni	No	Sì
VFX	No	Sì
Bot	Funzioni di scoring	Orchestrazione iniziale
Test	Sì	Functional Test

Gameplay Framework

Classe	Responsabilità nel progetto
ARTGameMode	Regole della partita, spawn e condizioni di vittoria

Classe	Responsabilità nel progetto
ARTGameState	Turno corrente, fase e punteggio
ARTPlayerController	Input, selezione e pianificazione
ARTCameraPawn	Camera tattica
ARTUnit	Unità combattente
ARTTurnManager	Raccolta e risoluzione delle azioni
URTAbilityData	Definizione data-driven di un'abilità
URTGridLibrary	Conversioni e funzioni della griglia
URCombatLibrary	Calcoli puri di danno e copertura
WBP_CombatHUD	Interfaccia principale

La separazione GameMode/GameState è utile anche prima del multiplayer: il GameMode rappresenta le regole autorevoli, mentre il GameState contiene lo stato condivisibile della partita. Questa struttura segue le convenzioni del Gameplay Framework di Unreal. ⁴

Struttura delle cartelle

```

ReactorTactics/
├── Build/
│   └── Scripts/
├── Config/
├── Content/
│   └── ReactorTactics/
│       ├── Art/
│       │   ├── Audio/
│       │   ├── Materials/
│       │   ├── Meshes/
│       │   ├── Textures/
│       │   └── VFX/
│       ├── Characters/
│       │   ├── Animations/
│       │   ├── Blueprints/
│       │   └── Data/
│       ├── Combat/
│       │   ├── Abilities/
│       │   ├── Phases/
│       │   ├── Status/
│       │   └── Targeting/
│       ├── Core/
│       ├── Grid/
│       ├── Maps/
│       ├── Tests/
│       └── UI/

```

```

├─ Plugins/
├─ Scripts/
├─ Source/
│   └─ ReactorTactics/
│       ├── Combat/
│       ├── Core/
│       ├── Grid/
│       ├── Tests/
│       └─ UI/

```

Convenzioni dei nomi

Prefisso	Tipo	Esempio
BP_	Blueprint Actor/Class	BP_RTUnit
WBP_	Widget Blueprint	WBP_CombatHUD
BPI_	Blueprint Interface	BPI_Selectable
DA_	Data Asset	DA_Ability_RifleShot
DT_	Data Table	DT_UnitDefinitions
IA_	Input Action	IA_Select
IMC_	Input Mapping Context	IMC_Tactical
L_	Level	L_Arena
M_	Material	M_GridTile
MI_	Material Instance	MI_GridTile_Reachable
T_	Texture	T_Ability_RifleShot
NS_	Niagara System	NS_Hit
S_	Sound	S_HitConfirm

Regole per evitare graph ingestibili

- Un Event Graph non dovrebbe diventare un'unica ragnatela.
- Trasforma blocchi ripetuti in funzioni.
- Usa componenti per responsabilità isolate.
- Evita `Event Tick` per la logica dei turni.
- Usa delegate, eventi, timer e state machine.
- Non inserire valori di bilanciamento direttamente nei nodi.
- Conserva salute, danni, range e cooldown nei Data Asset.
- Le animazioni ascoltano il resolver; non decidono il risultato.
- I widget leggono lo stato; non devono diventare il database del gioco.
- Compila e salva dopo ogni modifica strutturale.

Gameplay Tags

I Gameplay Tags sono etichette gerarchiche rappresentate da `FGameplayTag`, configurabili nelle Project Settings, nei file `.ini` o tramite Data Table. Sono adatti a rappresentare concetti come `Status.Rooted`, `Ability.Dash` o `Unit.Team.Enemy`. ¹⁶

Schema iniziale:

```
Ability.Prepare
Ability.Dash
Ability.Blast
Ability.Move

Status.Shielded
Status.Rooted
Status.Slowed
Status.Revealed

Unit.Team.Player
Unit.Team.Enemy

Target.Self
Target.Ally
Target.Enemy
Target.Ground
```

Per il tutorial non useremo inizialmente il Gameplay Ability System. GAS offre abilità, costi, cooldown ed effetti ed è molto adatto a giochi con molti poteri e status, ma introduce Ability System Component, Gameplay Effect, Attribute Set e prediction: troppo materiale prima di avere una vertical slice funzionante. ¹⁷

Programma delle lezioni

Indice

Lezione	Argomento	Risultato	Tempo
1	Ambiente	Toolchain funzionante	3-5 h
2	Progetto e Git	Repository iniziale	2-3 h
3	Fondamenti Unreal C++	Prima classe esposta	3-4 h
4	Arena e camera	Mappa navigabile	3-5 h
5	Enhanced Input	Controlli tattici	3-4 h
6	Selezione	Unità cliccabile	3-4 h
7	Griglia logica	Coordinate cella/mondo	4-6 h
8	Griglia visuale	Hover e celle colorate	4-5 h
9	Unità e squadre	Statistiche e team	4-5 h

Lezione	Argomento	Risultato	Tempo
10	Movimento	Range e percorso	5-7 h
11	Azioni pianificate	Comandi salvati come dati	4-6 h
12	Turn state machine	Sequenza delle fasi	5-7 h
13	Timer e lock-in	Decision Mode completa	4-6 h
14	Risoluzione movimento	Movimento simultaneo	5-8 h
15	Targeting	Linea, cono e area	6-8 h
16	Danni e scudi	Combattimento base	4-6 h
17	Abilità data-driven	Tre abilità configurabili	5-7 h
18	Cooldown ed energia	Ultimate e costi	4-6 h
19	Status e tag	Root, Slow e Shield	5-7 h
20	Linea di vista	Ostacoli e copertura	6-8 h
21	Bot	Avversario funzionante	6-9 h
22	HUD	Interfaccia completa	5-8 h
23	Presentazione	Animazioni, VFX e audio	5-8 h
24	Regole partita	Vittoria e riavvio	4-6 h
25	Salvataggio e test	Test e opzioni	5-8 h
26	Packaging e CI	Build distribuibile	5-8 h

Lezione uno — Preparare la workstation · tre-cinque ore

Obiettivo: avere Unreal, Visual Studio, Git e Git LFS installati e verificati.

Passi

1. Installa Epic Games Launcher.
2. Installa Unreal Engine 5.8.x con Starter Content.
3. Installa Visual Studio Community.
4. Seleziona il workload `Game development with C++`.
5. Installa Git e Git LFS.
6. Esegui:

```
git --version
git lfs version
```

1. Avvia Unreal una prima volta.
2. Crea un progetto C++ di prova.
3. Crea una classe `Actor`.
4. Compila da Visual Studio.

Snippet: non è ancora necessario codice gameplay. Il test è la compilazione di una classe vuota.

Equivalente Blueprint: crea un Blueprint Actor e posizionalo nel livello; serve a verificare il Content Browser.

Visuale: [Installare Visual Studio Tools per Unreal Engine](#).

Errore comune: installare solo il workload `.NET desktop development`. Unreal richiede la toolchain C++.

Esercizio: annota in `SETUP.md` versioni di Windows, Unreal, Visual Studio, driver GPU e Git.

Lezione due — Creare il progetto e il repository · due-tre ore

Obiettivo: creare `ReactorTactics`, la struttura di cartelle e il primo commit.

Passi

1. Crea un progetto Blank C++.
2. Attiva Starter Content.
3. Crea `Content/ReactorTactics`.
4. Crea le sottocartelle previste.
5. Inizializza Git.
6. Configura Git LFS per `.uasset` e `.umap`.
7. Crea `.gitignore`.
8. Salva il livello iniziale come:

```
Content/ReactorTactics/Maps/L_Arena
```

1. Impostalo come Editor Startup Map e Game Default Map.
2. Esegui il primo commit.

```
git add .
git commit -m "Initialize Reactor Tactics project"
```

Blueprint: crea `BP_ProjectMarker`, aggiungi un componente `Text Render` e visualizza `Reactor Tactics`.

Visuale: [Content Browser di Unreal Engine](#).

Errore comune: spostare file `.uasset` da Esplora File. Gli asset devono essere spostati dal Content Browser per aggiornare i riferimenti.

Esercizio: clona il repository in una seconda cartella e verifica che si apra.

Lezione tre — Tradurre C# in Unreal C++ · tre-quattro ore

Obiettivo: comprendere classi riflesse, header, source file e Blueprint subclass.

Passi

1. Crea una classe `Actor` chiamata `RTUnit`.
2. Aggiungi una proprietà `Health`.
3. Esponila al Blueprint.
4. Aggiungi una funzione `IsAlive`.
5. Compila.
6. Crea `BP_RTUnit` come figlio.
7. Modifica `Health` dal Details Panel.
8. Stampa il risultato di `IsAlive`.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category="Unit")
int32 Health = 100;

UFUNCTION(BlueprintPure, Category="Unit")
bool IsAlive() const
{
    return Health > 0;
}
```

Blueprint equivalente

```
Get Health
→ Greater Than 0
→ Return Value
```

Visuale: [Programmazione con C++ in Unreal Engine](#).

Errore comune: dimenticare `GENERATED_BODY()` o modificare un header usando Live Coding. Dopo modifiche riflesse esegui una compilazione completa.

Esercizio: aggiungi `MaxHealth` e una funzione BlueprintPure `GetHealthPercent`.

Lezione quattro — Costruire arena e camera · tre-cinque ore

Obiettivo: creare una piccola arena graybox e una camera tattica.

Passi

1. Inserisci un pavimento di circa `3000 × 3000` unità.
2. Crea muri con Cube scalati.
3. Aggiungi luci semplici.
4. Crea una classe Pawn Blueprint `BP_TacticalCamera`.
5. Aggiungi:
6. Scene Component;

7. Spring Arm;
8. Camera.
9. Inclina il braccio a circa `-55°`.
10. Imposta una distanza iniziale di circa `1800`.
11. Crea `BP_RTGameMode`.
12. Imposta `BP_TacticalCamera` come Default Pawn.
13. Avvia PIE.

Blueprint

```
Event BeginPlay
→ Set Show Mouse Cursor(true)
→ Set Input Mode Game And UI
```

Visuale: [Actors e componenti](#).

Errore comune: costruire già una mappa esteticamente complessa. Prima devono funzionare movimento e targeting.

Esercizio: aggiungi due zone rialzate e un corridoio centrale.

Lezione cinque — Configurare Enhanced Input · tre-quattro ore

Obiettivo: controllare camera, zoom, rotazione e click.

Passi

1. Crea `IMC_Tactical`.
2. Crea:
3. `IA_MoveCamera` ;
4. `IA_Zoom` ;
5. `IA_RotateCamera` ;
6. `IA_Select` ;
7. `IA_Cancel` ;
8. `IA_Confirm` .
9. Mappa WASD a `IA_MoveCamera` .
10. Mappa rotella a `IA_Zoom` .
11. Mappa Q/E a rotazione.
12. Mappa click sinistro a selezione.
13. Mappa destro o Esc a cancellazione.
14. Aggiungi il Mapping Context al BeginPlay.
15. Muovi il Pawn in base agli input.
16. Limita lo zoom.

Blueprint

```
IA_MoveCamera Triggered
→ Get Action Value
→ Make Vector
→ Add Actor World Offset
```

Visuale: [Enhanced Input](#).

I template moderni di Unreal usano Enhanced Input, che separa le azioni logiche dai dispositivi e dai mapping context. ¹⁸

Errore comune: moltiplicare lo spostamento per valori enormi senza `Delta Seconds`.

Esercizio: aggiungi pan della camera quando il cursore raggiunge il bordo dello schermo.

Lezione sei — Selezionare un'unità · tre-quattro ore

Obiettivo: individuare l'unità sotto il cursore e visualizzarne la selezione.

Passi

1. Aggiungi a `BP_RTUnit`:
2. Capsule;
3. Mesh;
4. Decal circolare.
5. Imposta il canale di collisione dell'unità.
6. Nascondi il Decal per default.
7. Nel PlayerController usa `Get Hit Result Under Cursor`.
8. Controlla se l'Actor implementa `BPI_Selectable`.
9. Deseleziona l'unità precedente.
10. Seleziona quella nuova.
11. Mostra il Decal.
12. Blocca la selezione delle unità nemiche.
13. Stampa il nome dell'unità.

Blueprint

```
IA_Select
→ Get Hit Result Under Cursor by Channel
→ Hit Actor
→ Does Implement Interface
→ Select
```

Visuale: [Blueprint Interface](#).

Errore comune: il trace colpisce il pavimento perché il mesh dell'unità non blocca il canale corretto.

Esercizio: cambia il colore del Decal in base alla squadra.

Lezione sette — Creare la griglia logica · quattro-sei ore

Obiettivo: convertire coordinate del mondo in celle e viceversa.

Passi

1. Definisci `CellSize = 200`.
2. Crea `URTGridLibrary`.
3. Implementa `WorldToGrid`.
4. Implementa `GridToWorld`.
5. Esponi entrambe a Blueprint.
6. Posiziona un Cube usando coordinate cella.
7. Muovi il Cube tra `(0,0)`, `(1,0)` e `(1,1)`.
8. Stampa la cella sotto il cursore.
9. Aggiungi un offset `GridOrigin`.
10. Verifica valori negativi.

```
FIntPoint Cell(  
    FMath::RoundToInt(World.X / CellSize),  
    FMath::RoundToInt(World.Y / CellSize)  
);
```

Blueprint equivalente

```
World Location X / Cell Size  
→ Round  
→ Grid X  
  
World Location Y / Cell Size  
→ Round  
→ Grid Y
```

Visuale: [Blueprint Function Libraries](#).

Errore comune: usare `Floor` in una funzione e `Round` nell'altra, generando conversioni incoerenti.

Esercizio: verifica con cinque celle che `WorldToGrid(GridToWorld(Cell)) == Cell`.

Lezione otto — Visualizzare e interrogare la griglia · quattro-cinque ore

Obiettivo: mostrare celle, hover, celle libere e occupate.

Passi

1. Crea `BP_GridManager`.
2. Definisci larghezza e altezza.

3. Genera una griglia `10 × 10`.
4. Per il prototipo usa Instanced Static Mesh o Decal.
5. Salva le celle in una mappa dati.
6. Esegui un trace dal cursore al pavimento.
7. Converti l'impatto in cella.
8. Evidenzia la cella corrente.
9. Aggiungi lo stato `Blocked`.
10. Colora diversamente le celle occupate.

Blueprint

```
Hit Location
→ World To Grid
→ Is Valid Cell
→ Update Hover Instance
```

Visuale: [Instanced Static Mesh Component](#).

Errore comune: creare cento Actor indipendenti e aggiungere Tick a ciascuno.

Esercizio: crea una forma irregolare disabilitando dieci celle.

Lezione nove — Definire unità e squadre · quattro-cinque ore

Obiettivo: creare statistiche, identificazione squadra e dati iniziali.

Passi

1. Crea `ERTTeam`.
2. Aggiungi `Player` ed `Enemy`.
3. Crea `FRTUnitStats`.
4. Inserisci:
5. `MaxHealth`;
6. `MoveRange`;
7. `Energy`;
8. `Armor`.
9. Aggiungi `UnitId`.
10. Aggiungi `Team`.
11. Crea due Blueprint figli:
12. `BP_Unit_Ranger`;
13. `BP_Unit_Guardian`.
14. Assegna valori diversi.
15. Posiziona due unità per squadra.

16. Mostra una barra salute provvisoria.

```
UENUM(BlueprintType)
enum class ERTTeam : uint8
{
    Player,
    Enemy
};
```

Blueprint equivalente: crea un Blueprint Enum con `Player` ed `Enemy`; nel percorso definitivo usa però l'enum C++ per condividerlo con resolver e test.

Visuale: [Struct e Data Table](#).

Errore comune: usare il colore del materiale per determinare la squadra. Il colore deve rappresentare i dati, non sostituirli.

Esercizio: impedisce al giocatore di selezionare `Enemy`.

Lezione dieci — Movimento e anteprima percorso · cinque-sette ore

Obiettivo: mostrare dove un'unità può muoversi e pianificare una destinazione.

Passi

1. Recupera la cella dell'unità.
2. Calcola la distanza Manhattan:

```
Distance = Abs(A.X - B.X) + Abs(A.Y - B.Y);
```

1. Mostra celle con distanza minore o uguale a `MoveRange`.
2. Escludi celle bloccate.
3. Escludi celle occupate.
4. Clicca una cella valida.
5. Salva la destinazione.
6. Mostra una spline o una sequenza di frecce.
7. Click destro cancella.
8. Non muovere ancora l'unità: stai solo pianificando.

Blueprint

```
Abs(TargetX - StartX)
+ Abs(TargetY - StartY)
≤ MoveRange
```

Visuale: [Spline Components](#).

Errore comune: spostare subito l'unità durante la Decision Mode.

Esercizio: sostituisci la distanza semplice con una breadth-first search che aggira i muri.

Lezione undici — Rappresentare le azioni come dati · quattro-sei ore

Obiettivo: creare una struttura unica per le azioni pianificate.

Passi

1. Crea enum `ERTCombatPhase`.
2. Crea struct `FRTPlannedAction`.
3. Aggiungi:
4. Unit;
5. Ability;
6. Target Actor;
7. Target Location;
8. Destination;
9. Phase;
10. Locked.
11. Aggiungi un'azione pianificata a ogni unità.
12. Crea `ClearPlannedAction`.
13. Crea `HasValidAction`.
14. Visualizza l'azione nella UI.
15. Non conservare questi dati dentro il Widget.
16. Verifica che la cancellazione ripristini tutto.
17. Registra l'azione nel log.

```
USTRUCT(BlueprintType)
struct FRTPlannedAction
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    ERTCombatPhase Phase = ERTCombatPhase::Blast;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FVector TargetLocation = FVector::ZeroVector;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    bool bLocked = false;
};
```

Blueprint equivalente: crea una Blueprint Struct con gli stessi campi e usa `Make RTPlannedAction`.

Visuale: [Struct Variables in Blueprints](#).

Errore comune: salvare direttamente riferimenti ai widget nell'azione.

Esercizio: serializza l'azione in una stringa leggibile per il debug.

Lezione dodici — Costruire la state machine del turno · cinque-sette ore

Obiettivo: eseguire sempre lo stesso ordine di fasi.

Passi

1. Crea `ARTTurnManager` .
2. Aggiungi `CurrentTurn` .
3. Aggiungi `CurrentPhase` .
4. Definisci:
5. `Decision`;
6. `Prep`;
7. `Dash`;
8. `Blast`;
9. `Move`;
10. `Cleanup`.
11. Crea `StartDecision` .
12. Crea `ResolveCurrentPhase` .
13. Crea `AdvancePhase` .
14. Esponi `OnPhaseChanged` .
15. Stampa ogni transizione.
16. Dopo `Cleanup` incrementa il turno.

```
Decision
→ Prep
→ Dash
→ Blast
→ Move
→ Cleanup
→ Decision
```

Blueprint

```
Switch on ERTCombatPhase
→ Execute Phase
→ Set Timer
→ Advance Phase
```

Visuale: [Event Dispatchers](#).

Errore comune: affidare l'avanzamento della fase alla fine di una singola animazione.

Esercizio: crea un test manuale con un pulsante `Advance Phase` .

Lezione tredici — Timer e lock-in · quattro-sei ore

Obiettivo: completare la modalità di pianificazione.

Passi

1. Imposta un timer di prova di trenta secondi.
2. Mostralo nella UI.
3. Riducilo ogni secondo con un Timer, non con Tick se possibile.
4. Aggiungi pulsante `Conferma`.
5. Quando il giocatore conferma:
6. valida le azioni;
7. assegna azione Wait alle unità senza comando;
8. imposta `bLocked`;
9. disabilita input di pianificazione.
10. Fai pianificare il bot.
11. Quando tutti sono pronti avvia Prep.
12. Alla scadenza esegui lock automatico.
13. Aggiungi feedback audio negli ultimi cinque secondi.
14. Per il prototipo usa quindici-trenta secondi; bilancia dopo.

Blueprint

```
Confirm Button
→ Validate Team Actions
→ Lock Team
→ Are All Teams Locked?
→ Start Resolution
```

Visuale: [Using Timers](#).

Errore comune: il timer continua durante la Resolution Mode.

Esercizio: aggiungi pausa e ripresa del timer per il debugging offline.

Lezione quattordici — Risolvere il movimento simultaneo · cinque-otto ore

Obiettivo: gestire più unità che vogliono raggiungere celle in conflitto.

Passi

1. Raccogli tutte le azioni Move.
2. Crea uno snapshot delle posizioni iniziali.
3. Calcola tutti i percorsi prima di animare.
4. Definisci una regola per due unità sulla stessa cella.
5. Per il tutorial usa:

6. cella contesa → entrambe si fermano prima;
7. scambio diretto → consentito;
8. cella occupata da unità ferma → movimento bloccato.
9. Calcola lo stato finale.
10. Aggiorna l'occupazione logica.
11. Riproduci l'animazione.
12. Concludi la fase solo quando tutte le animazioni sono terminate.
13. Mantieni separati calcolo e presentazione.

Pseudo-codice

```
Snapshot  
→ Collect Intents  
→ Resolve Conflicts  
→ Produce Final Positions  
→ Animate Results
```

Blueprint equivalente: una funzione `ResolveMoveIntents` restituisce un array di risultati; una seconda funzione `PlayMoveResults` anima le unità.

Visuale: [Arrays in Blueprint](#).

Errore comune: aggiornare la prima unità e usare la sua nuova posizione per calcolare la seconda.

Esercizio: crea tre scenari di conflitto e annota il risultato atteso.

Lezione quindici — Targeting di linea, cono e area · sei-otto ore

Obiettivo: creare le forme di targeting più comuni.

Passi

1. Crea enum `ERTTargetShape`.
2. Aggiungi:
3. Self;
4. Single;
5. Line;
6. Cone;
7. Circle.
8. Per `Single`, verifica distanza e squadra.
9. Per `Circle`, usa distanza dal centro.
10. Per `Line`, proietta la cella sul vettore direzione.
11. Per `Cone`, usa dot product e angolo.
12. Visualizza la forma con Decal o mesh trasparenti.
13. Evidenzia i bersagli validi.

14. Salva centro, direzione e target nell'azione.
15. Aggiungi preview del danno.

C++ minimale

```
const FVector ToTarget = (Target - Origin).GetSafeNormal();
const float Alignment = FVector::DotProduct(Forward, ToTarget);
const bool bInsideCone = Alignment >=
FMath::Cos(FMath::DegreesToRadians(HalfAngle));
```

Blueprint equivalente

```
Forward Vector
+ Direction To Target
→ Dot Product
→ Compare with Cos(Half Angle)
```

Visuale: [Vector Math](#).

Errore comune: confrontare direttamente il dot product con un angolo espresso in gradi.

Esercizio: implementa un attacco a linea che colpisce il primo bersaglio.

Lezione sedici — Danni, scudi e morte · quattro-sei ore

Obiettivo: creare un modello di combattimento prevedibile.

Passi

1. Aggiungi `Health`, `MaxHealth` e `Shield`.
2. Crea `ApplyDamage`.
3. Lo scudo assorbe per primo.
4. Il danno rimanente riduce Health.
5. Health non scende sotto zero.
6. Emetti `OnHealthChanged`.
7. Emetti `OnUnitDefeated`.
8. Non distruggere immediatamente l'Actor durante Blast.
9. Segna l'unità come `PendingDefeat`.
10. Rimuovila durante Cleanup.

```
const int32 Absorbed = FMath::Min(Shield, Damage);
Shield -= Absorbed;

const int32 Remaining = Damage - Absorbed;
Health = FMath::Max(0, Health - Remaining);
```

Blueprint equivalente

```
Min(Shield, Damage)
→ New Shield
→ Remaining Damage
→ Clamp Health
```

Visuale: [Math Expression Node](#).

Errore comune: eliminare subito l'unità e impedirle di completare un'azione già pianificata nella stessa fase.

Esercizio: aggiungi una funzione `Heal` che non supera `MaxHealth`.

Lezione diciassette — Abilità data-driven · cinque-sette ore

Obiettivo: creare nuove abilità senza modificare il resolver.

Passi

1. Crea `URTAbilityData : UPrimaryDataAsset`.
2. Aggiungi:
3. Name;
4. Icon;
5. Phase;
6. Shape;
7. Range;
8. Radius;
9. Damage;
10. Cooldown;
11. EnergyCost;
12. TargetRules.
13. Crea tre Data Asset:
14. `DA_RifleShot` ;
15. `DA_EnergyShield` ;
16. `DA_Dash` .
17. Assegna le abilità alle unità.
18. Genera automaticamente i pulsanti UI.
19. Il resolver legge i dati.
20. Aggiungi un evento Blueprint per VFX personalizzati.
21. Non creare una classe C++ diversa per ogni attacco semplice.
22. Mantieni una `AbilityId` .
23. Verifica i dati all'avvio.

C++ concettuale

```
UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
int32 Damage = 20;

UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
int32 Range = 5;
```

Blueprint equivalente: un Blueprint Primary Data Asset contenente gli stessi campi.

Visuale: [Data Assets](#).

Errore comune: creare variabili duplicate nel Widget, nell'unità e nel Data Asset.

Esercizio: duplica `DA_RifleShot` e crea una variante a cono senza cambiare codice.

Lezione diciotto — Cooldown, energia e ultimate · quattro-sei ore

Obiettivo: limitare l'uso delle abilità e creare una risorsa tattica.

Passi

1. Aggiungi un array di cooldown correnti.
2. Quando un'abilità viene usata, imposta il cooldown.
3. Durante Cleanup riduci i cooldown.
4. Aggiungi energia massima cento.
5. Guadagna energia:
6. a ogni turno;
7. quando un attacco colpisce.
8. Blocca l'ultimate sotto cento.
9. Quando viene usata, consuma cento.
10. Aggiorna UI ed effetti.
11. Mostra tooltip con turni rimanenti.
12. Scrivi un test per evitare cooldown negativi.

Blueprint

```
On Cleanup
→ For Each Cooldown
→ Max(Current - 1, 0)
```

Atlas Reactor utilizzava energia ottenuta nel tempo e attraverso gli attacchi, rendendo l'ultimate disponibile a cento energia; qui il concetto viene riutilizzato con valori originali da bilanciare. ¹

Visuale: [Progress Bar UMG](#).

Errore comune: decrementare il cooldown immediatamente nello stesso turno in cui viene applicato.

Esercizio: crea un'abilità che costa cinquanta energia ma non è un'ultimate.

Lezione diciannove — Status effect e Gameplay Tags · cinque-sette ore

Obiettivo: implementare Root, Slow, Shield e Reveal.

Passi

1. Configura i Gameplay Tags.
2. Crea `FRTStatusEffect`.
3. Aggiungi:
4. Tag;
5. Duration;
6. Magnitude;
7. Source.
8. Crea un array di status sull'unità.
9. Implementa `HasStatus`.
10. `Status.Rooted` impedisce Move.
11. `Status.Slowed` riduce range.
12. `Status.Shielded` aggiunge scudo.
13. Durante Cleanup riduci la durata.
14. Rimuovi gli status scaduti.

```
Status.Rooted  
Status.Slowed  
Status.Shielded  
Status.Revealed
```

Blueprint

```
Has Matching Gameplay Tag  
→ Branch  
→ Modify Available Actions
```

Visuale: [Gameplay Tags](#).

Errore comune: confrontare i tag convertendoli in stringhe.

Esercizio: crea `Status.Haste` che aumenta il movimento di due celle.

Lezione venti — Linea di vista e copertura · sei-otto ore

Obiettivo: impedire attacchi attraverso muri e applicare una copertura semplice.

Passi

1. Crea un canale collisione `VisibilityCombat`.
2. Esegui un Line Trace tra attaccante e bersaglio.
3. Se colpisce un muro, il bersaglio non è visibile.
4. Aggiungi punti di campionamento su torso e testa.
5. Definisci copertura leggera e pesante.
6. Per la vertical slice usa:
7. nessuna copertura: danno pieno;
8. copertura: riduzione fissa;
9. muro alto: attacco bloccato.
10. Mostra l'icona di copertura.
11. Calcola la copertura durante preview e risoluzione.
12. Non usare la camera per determinare la visibilità.
13. Scrivi test su una configurazione nota.

Blueprint

```
Line Trace By Channel  
→ Break Hit Result  
→ Is Hit Actor Target?  
→ Visible
```

Visuale: [Line Trace by Channel](#).

L'originale distingueva terreno alto che bloccava linea di vista e terreno basso che offriva copertura; il tutorial adotta una versione semplificata dello stesso tipo di decisione spaziale. ¹

Errore comune: il trace colpisce il mesh dell'attaccante. Ignora sempre il source Actor.

Esercizio: evidenzia le celle visibili dall'unità selezionata.

Lezione ventuno — Creare il bot · sei-nove ore

Obiettivo: permettere alla squadra nemica di pianificare azioni complete.

Passi

1. Per ogni unità nemica genera azioni candidate.
2. Trova bersagli visibili.
3. Calcola uno score.
4. Esempio:

```
Score =  
DamageExpected * 3  
+ KillChance * 100
```

- Distance * 2
- DangerAtDestination * 4

1. Aggiungi candidati Move.
2. Aggiungi candidato Shield se la salute è bassa.
3. Scarta azioni non valide.
4. Scegli il candidato con score maggiore.
5. Aggiungi una piccola variazione casuale.
6. Salva il seed casuale per riprodurre bug.
7. Fai lock-in al bot.
8. Visualizza in debug il motivo della scelta.

Blueprint

```
Generate Candidate Actions
→ For Each
→ Calculate Score
→ Keep Best
→ Set Planned Action
```

Visuale: [Behavior Trees](#).

Per il primo bot non è necessario un Behavior Tree: un sistema utility basato su candidati e score corrisponde meglio a una decisione discreta di turno. Il Behavior Tree potrà essere usato in seguito per obiettivi strategici più ampi.

Errore comune: far conoscere al bot dati che dovrebbero essere nascosti al giocatore.

Esercizio: aggiungi tre livelli di difficoltà modificando profondità di valutazione e rumore.

Lezione ventidue — Costruire HUD e combat log · cinque-otto ore

Obiettivo: rendere leggibile lo stato della partita.

Passi

1. Crea `WBP_CombatHUD`.
2. Aggiungi:
3. turno;
4. fase;
5. timer;
6. unità selezionata;
7. salute;
8. energia;
9. pulsanti abilità;
10. pulsante lock-in;

11. combat log.
12. Crea `WBP_AbilityButton`.
13. Genera i pulsanti dai Data Asset.
14. Aggiorna la UI tramite eventi.
15. Mostra cooldown e costo.
16. Disabilita pulsanti non validi.
17. Aggiungi tooltip.
18. Aggiungi indicatori sopra le unità.
19. Testa a 1920×1080 e 2560×1440.

Blueprint

```
On Unit Selected
→ Update Unit Panel

On Phase Changed
→ Update Phase Label

On Health Changed
→ Update Health Bar
```

UMG è il framework principale di Unreal per costruire interfacce tramite Widget Blueprint, pannelli, controlli e binding/eventi. ¹⁹

Visuale: [Creare UI con UMG](#).

Errore comune: usare binding UMG eseguiti ogni frame per tutte le proprietà.

Esercizio: aggiungi un pannello che mostra l'azione pianificata di ogni alleato.

Lezione ventitré — Animazioni, effetti e audio · cinque-otto ore

Obiettivo: trasformare il resolver in un'esperienza leggibile.

Passi

1. Importa o usa un personaggio placeholder.
2. Configura idle, run e hit.
3. Crea un Animation Blueprint minimale.
4. Il movimento logico produce un evento visuale.
5. Crea Niagara per:
 6. colpo;
 7. scudo;
 8. selezione;
 9. cella target.
10. Aggiungi suoni provvisori.
11. Usa camera shake molto leggero.

12. Aggiungi pausa breve tra le fasi.
13. Permetti velocità di risoluzione `1x` e `2x`.
14. Mantieni gli eventi cosmetici separati dai risultati.

Blueprint

```
On Combat Event
→ Switch Event Type
→ Play Montage
→ Spawn Niagara
→ Play Sound
```

Visuale: [Niagara Overview](#).

Errore comune: applicare il danno in un Anim Notify. Il Notify deve mostrare il colpo, non determinarne il risultato.

Esercizio: disabilita tutte le animazioni e verifica che il risultato del turno resti identico.

Lezione ventiquattro — Regole, vittoria e riavvio · quattro-sei ore

Obiettivo: trasformare il sandbox in una partita completa.

Passi

1. Il GameMode registra le unità.
2. Durante Cleanup conta le unità vive.
3. Se una squadra non ha unità vive, termina la partita.
4. Blocca input e timer.
5. Mostra `WBP_MatchResult`.
6. Aggiungi:
7. Restart;
8. Main Menu;
9. Quit.
10. Registra durata e numero di turni.
11. Aggiungi limite opzionale di quindici turni.
12. In caso di parità mostra Draw.
13. Testa morte simultanea.

Blueprint

```
Cleanup Complete
→ Count Alive Player Units
→ Count Alive Enemy Units
→ Evaluate Match Result
```

Visuale: [Game Mode and Game State](#).

Errore comune: controllare la vittoria nel momento esatto in cui una singola unità riceve danno.

Esercizio: aggiungi una variante "prima squadra a tre eliminazioni".

Lezione venticinque — Salvataggio, debugging e test · cinque-otto ore

Obiettivo: salvare opzioni e proteggere la logica dai regressioni.

Passi

1. Crea `URTSaveGame`.
2. Salva:
3. volume;
4. risoluzione;
5. velocità della camera;
6. velocità di risoluzione;
7. ultimo livello.
8. Aggiungi log categorizzati.
9. Crea test per:
10. conversione griglia;
11. danni e scudi;
12. cooldown;
13. transizioni delle fasi.
14. Crea un Functional Test Blueprint.
15. Esegui i test da Session Frontend.
16. Esegui da command line.
17. Esporta report JSON.
18. Correggi warning Blueprint.
19. Verifica che non esistano errori di asset mancanti.

```
UE_LOG(LogTemp, Log, TEXT("Turn %d, phase %d"), Turn,
static_cast<int32>(Phase));
```

Blueprint equivalente

```
Print String
```

Usa `Print String` solo temporaneamente. Per problemi riproducibili preferisci Output Log e categorie C++.

L'Automation Framework di Unreal supporta test unitari legati all'engine, feature test e content stress test; i test possono essere eseguiti dall'Editor o da command line. ²⁰

Visuale: [Automation Test Framework](#).

Errore comune: testare soltanto l'animazione e non il risultato logico.

Esercizio: crea un test che verifica l'ordine completo

Decision → Prep → Dash → Blast → Move → Cleanup .

Lezione ventisei — Packaging e integrazione continua · cinque-otto ore

Obiettivo: produrre una build Windows ripetibile.

Passi

1. Imposta `L_Arena` come Game Default Map.
2. Aggiungi la mappa alla lista di cooking.
3. Crea una build Development dall'Editor.
4. Esegui la build fuori dall'Editor.
5. Verifica input, font, audio e caricamento asset.
6. Crea una build Shipping.
7. Aggiungi script PowerShell per test.
8. Aggiungi script PowerShell per packaging.
9. Configura un runner CI Windows.
10. Archivia la cartella della build.

Comando minimale

```
& "C:\Program Files\Epic Games\UE_5.8\Engine\Build\BatchFiles\RunUAT.bat" `
  BuildCookRun `
  -project="D:\Dev\ReactorTactics\ReactorTactics.uproject" `
  -noP4 `
  -platform=Win64 `
  -clientconfig=Development `
  -build `
  -cook `
  -allmaps `
  -stage `
  -pak `
  -archive `
  -archivedirectory="D:\Builds\ReactorTactics" `
  -utf8output
```

Il packaging attraversa le fasi Build, Cook, Stage e Package; BuildCookRun è il comando UAT che orchestra tali operazioni. ²¹

Visuale: [Packaging Your Project](#).

Errore comune: build con schermata nera perché non è stata configurata la Game Default Map.

Esercizio finale: consegna la build a una persona che non possiede Unreal e raccogli i primi cinque problemi incontrati.

Codice minimo ed equivalenti Blueprint

Enum delle fasi

Crea `RTCombatTypes.h`:

```
#pragma once

#include "CoreMinimal.h"
#include "RTCombatTypes.generated.h"

UENUM(BlueprintType)
enum class ERTCombatPhase : uint8
{
    Decision,
    Prep,
    Dash,
    Blast,
    Move,
    Cleanup
};
```

Blueprint:

```
Content Browser
→ Add
→ Blueprints
→ Enumeration
```

Per il progetto finale preferisci l'enum C++, perché sarà usato da resolver, GameState e test.

Struttura dell'azione pianificata

Nello stesso file:

```
USTRUCT(BlueprintType)
struct FRTPlannedAction
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    ERTCombatPhase Phase = ERTCombatPhase::Blast;
```

```

UPROPERTY(EditAnywhere, BlueprintReadWrite)
TObjectPtr<AActor> Target = nullptr;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
FVector TargetLocation = FVector::ZeroVector;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
FIntPoint DestinationCell = FIntPoint::ZeroValue;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
FName AbilityId = NAME_None;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
bool bLocked = false;
};

```

Equivalente Blueprint:

```

Make FRTPlannedAction
├─ Phase
├─ Target
├─ Target Location
├─ Destination Cell
├─ Ability Id
└─ Locked

```

Libreria della griglia

RTGridLibrary.h:

```

#pragma once

#include "CoreMinimal.h"
#include "Kismet/BlueprintFunctionLibrary.h"
#include "RTGridLibrary.generated.h"

UCLASS()
class REACTORTACTICS_API URTGridLibrary
: public UBlueprintFunctionLibrary
{
    GENERATED_BODY()

public:
    UFUNCTION(BlueprintPure, Category="RT|Grid")
    static FIntPoint WorldToGrid(
        const FVector& WorldLocation,
        const FVector& GridOrigin,

```

```

        float CellSize
    );

    UFUNCTION(BlueprintPure, Category="RT|Grid")
    static FVector GridToWorld(
        FIntPoint Cell,
        const FVector& GridOrigin,
        float CellSize,
        float Height = 0.0f
    );

    UFUNCTION(BlueprintPure, Category="RT|Grid")
    static int32 ManhattanDistance(FIntPoint A, FIntPoint B);
};

```

RTGridLibrary.cpp:

```

#include "Grid/RTGridLibrary.h"

FIntPoint URTGridLibrary::WorldToGrid(
    const FVector& WorldLocation,
    const FVector& GridOrigin,
    const float CellSize)
{
    if (CellSize <= 0.0f)
    {
        return FIntPoint::ZeroValue;
    }

    const FVector Local = WorldLocation - GridOrigin;

    return FIntPoint(
        FMath::RoundToInt(Local.X / CellSize),
        FMath::RoundToInt(Local.Y / CellSize)
    );
}

FVector URTGridLibrary::GridToWorld(
    const FIntPoint Cell,
    const FVector& GridOrigin,
    const float CellSize,
    const float Height)
{
    return GridOrigin + FVector(
        Cell.X * CellSize,
        Cell.Y * CellSize,
        Height
    );
}

```

```
int32 URTGridLibrary::ManhattanDistance(
    const FIntPoint A,
    const FIntPoint B)
{
    return FMath::Abs(A.X - B.X)
        + FMath::Abs(A.Y - B.Y);
}
```

Blueprint equivalente:

```
World Location - Grid Origin
→ Break Vector
→ X / Cell Size
→ Round
→ Grid X

→ Y / Cell Size
→ Round
→ Grid Y
```

Unità combattente minimale

RTUnit.h:

```
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/Actor.h"
#include "RTCombatTypes.h"
#include "RTUnit.generated.h"

DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams(
    FRTOnHealthChanged,
    int32, NewHealth,
    int32, NewShield
);

UCLASS()
class REACTORTACTICS_API ARTUnit : public AActor
{
    GENERATED_BODY()

public:
    ARTUnit();

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category="Unit")
    int32 MaxHealth = 100;
```



```

UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category="Unit")
int32 Health = 100;

UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category="Unit")
int32 Shield = 0;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category="Unit")
FRTPlannedAction PlannedAction;

UPROPERTY(BlueprintAssignable)
FRTOnHealthChanged OnHealthChanged;

UFUNCTION(BlueprintCallable, Category="Unit")
void ApplyCombatDamage(int32 Damage);

UFUNCTION(BlueprintPure, Category="Unit")
bool IsAlive() const
{
    return Health > 0;
}
};

```

RTUnit.cpp:

```

#include "Characters/RTUnit.h"

ARTUnit::ARTUnit()
{
    PrimaryActorTick.bCanEverTick = false;
}

void ARTUnit::ApplyCombatDamage(const int32 Damage)
{
    if (Damage <= 0 || Health <= 0)
    {
        return;
    }

    const int32 Absorbed = FMath::Min(Shield, Damage);
    Shield -= Absorbed;

    const int32 Remaining = Damage - Absorbed;
    Health = FMath::Max(0, Health - Remaining);

    OnHealthChanged.Broadcast(Health, Shield);
}

```

Blueprint equivalente:

```
Apply Combat Damage
→ Damage > 0?
→ Absorbed = Min(Shield, Damage)
→ Shield = Shield - Absorbed
→ Remaining = Damage - Absorbed
→ Health = Max(0, Health - Remaining)
→ On Health Changed
```

Avanzamento delle fasi

Versione semplice:

```
void ARTTurnManager::AdvancePhase()
{
    switch (CurrentPhase)
    {
        case ERTCombatPhase::Decision:
            CurrentPhase = ERTCombatPhase::Prep;
            break;

        case ERTCombatPhase::Prep:
            CurrentPhase = ERTCombatPhase::Dash;
            break;

        case ERTCombatPhase::Dash:
            CurrentPhase = ERTCombatPhase::Blast;
            break;

        case ERTCombatPhase::Blast:
            CurrentPhase = ERTCombatPhase::Move;
            break;

        case ERTCombatPhase::Move:
            CurrentPhase = ERTCombatPhase::Cleanup;
            break;

        case ERTCombatPhase::Cleanup:
            ++CurrentTurn;
            CurrentPhase = ERTCombatPhase::Decision;
            break;
    }

    OnPhaseChanged.Broadcast(CurrentPhase);
}
```

Blueprint equivalente:

Switch on ERTCombatPhase

Decision → Set Prep

Prep → Set Dash

Dash → Set Blast

Blast → Set Move

Move → Set Cleanup

Cleanup → Increment Turn → Set Decision

→ Call OnPhaseChanged

Calcolo puro del danno

Quando possibile, separa il calcolo dall'Actor:

```
USTRUCT(BlueprintType)
struct FRTDamageResult
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly)
    int32 NewHealth = 0;

    UPROPERTY(BlueprintReadOnly)
    int32 NewShield = 0;

    UPROPERTY(BlueprintReadOnly)
    int32 AppliedDamage = 0;
};
```

```
FRTDamageResult CalculateDamage(
    int32 Health,
    int32 Shield,
    int32 Damage);
```

Questo permette di testare il calcolo senza caricare un livello.

Test automatico minimo

Crea `RTGridTests.cpp`:

```
#if WITH_AUTOMATION_TESTS

#include "Misc/AutomationTest.h"
#include "Grid/RTGridLibrary.h"
```

```

IMPLEMENT_SIMPLE_AUTOMATION_TEST(
    FRTGridRoundTripTest,
    "ReactorTactics.Grid.RoundTrip",
    EAutomationTestFlags::EditorContext |
    EAutomationTestFlags::EngineFilter
)

bool FRTGridRoundTripTest::RunTest(const FString& Parameters)
{
    const FVector Origin = FVector::ZeroVector;
    const float CellSize = 200.0f;
    const FIntPoint ExpectedCell(3, -2);

    const FVector World =
        URTGridLibrary::GridToWorld(
            ExpectedCell,
            Origin,
            CellSize
        );

    const FIntPoint ActualCell =
        URTGridLibrary::WorldToGrid(
            World,
            Origin,
            CellSize
        );

    TestEqual(
        TEXT("Grid to world round trip"),
        ActualCell,
        ExpectedCell
    );

    return true;
}

#endif

```

Esecuzione da PowerShell:

```

& "C:\Program Files\Epic Games\UE_5.8\Engine\Binaries\Win64\UnrealEditor-
Cmd.exe" `
    "D:\Dev\ReactorTactics\ReactorTactics.uproject" `
    -unattended `
    -nop4 `
    -nullrhi `
    -ExecCmds="Automation RunTest ReactorTactics;Quit" `
    -ReportExportPath="D:\Builds\ReactorTactics\TestReports" `
    -utf8output

```

`Automation RunTest` può eseguire singoli test o intere gerarchie di nomi, mentre `ReportExportPath` salva il report con dettagli, eventi, durate e informazioni sul dispositivo. ²²

Asset, testing, debugging, build e CI

Strategia degli asset

Durante le prime venti lezioni usa:

- Cube;
- Cylinder;
- Capsule;
- materiali a tinta piatta;
- icone provvisorie;
- Starter Content;
- font inclusi o con licenza chiara.

Lo Starter Content contiene mesh, materiali, texture, particelle e suoni progettati per prototipare direttamente nei nuovi progetti. ²³

Dopo che il gameplay funziona, puoi introdurre asset gratuiti da Fab.

Risorsa	Uso suggerito	Nota
Starter Content	Graybox, materiali, audio	Incluso nell'installazione
Paragon Characters	Personaggi dimostrativi	Solo progetti Unreal, verificare la singola licenza
Paragon Minions	Unità placeholder	Non usare il marchio Paragon nel nome/ promozione
Stack O Bot	Studio di un progetto compatto	Buono per esplorazione
Lyra Starter Game	Architettura avanzata	Non copiarlo all'inizio

Risorse:

- [Paragon: Phase](#)
- [Paragon: Minions](#)
- [Paragon: Wraith](#)
- [Paragon: Crunch](#)

Le pagine Fab degli asset Paragon indicano restrizioni specifiche su marchi, nomi e promozione; per un prodotto originale è quindi importante leggere la licenza di ogni listing e non presentare il gioco come continuazione o ricostruzione di Paragon. ²⁴

Regola degli asset

Non introdurre un asset finché non sai rispondere a queste domande:

1. È necessario per verificare il gameplay?
2. La licenza permette l'uso previsto?
3. È compatibile con la versione di Unreal?
4. Quanto pesa?
5. Ha skeleton e animazioni adatti?
6. Può essere sostituito senza cambiare la logica?

Checklist quotidiana di debugging

Controllo	Frequenza
Output Log senza errori rossi	Ogni sessione
Compile di tutti i Blueprint	Ogni sessione
Build C++ completa	Almeno una volta al giorno
Commit Git	Alla fine di una feature funzionante
Test della griglia	Dopo modifiche al movimento
Test danni	Dopo modifiche al combattimento
Test da build packaged	Ogni settimana
Verifica riferimenti asset	Prima di una release

Strumenti di debug

Per Blueprint:

- breakpoint sui nodi;
- Watch Value;
- Blueprint Debugger;
- `Print String` temporaneo;
- Visual Logger;
- Gameplay Debugger;
- Output Log.

Per C++:

```
UE_LOG(LogTemp, Warning, TEXT("Unit %s has no action"), *GetName());
```

Per condizioni recuperabili:

```
if (!ensure(Target != nullptr))  
{  
    return;  
}
```

Per condizioni che non devono mai essere false in sviluppo:

```
check(CellSize > 0.0f);
```

Usa `check` con moderazione: termina l'esecuzione nelle configurazioni in cui è attivo.

Visual Studio Tools for Unreal consente di osservare log, macro, riferimenti Blueprint e informazioni Blueprint durante il debugging del C++. ⁶

Bug tipici del sistema simultaneo

Sintomo	Causa probabile	Correzione
L'ordine cambia tra esecuzioni	Iterazione su collezione non ordinata	Ordina per UnitId
Unità morta perde l'azione	Rimozione troppo anticipata	Applica la morte in Cleanup
Movimento incoerente	Stato modificato durante il calcolo	Usa snapshot
Preview diversa dal risultato	Due implementazioni della formula	Condividi la stessa funzione
Timer continua in risoluzione	Timer non cancellato	ClearTimer al lock-in
VFX modifica il risultato	Logica in Anim Notify	Sposta la logica nel resolver
Bot vede attraverso muri	Scoring senza visibility check	Filtra candidati
Build non trova asset	Riferimento solo tramite stringa	Usa riferimenti soft o asset manager
Packaged build nera	Default Map assente	Configura Maps & Modes
Crash dopo modifica header	Live Coding su reflection	Chiudi Editor e ricompila

Strategia di test

La piramide consigliata è:

```
Molti test puri
  Griglia
  Danno
  Cooldown
  Status
  Targeting

Meno test di integrazione
  Resolver
```

Conflitti di movimento
Ordine delle fasi

Pochi test end-to-end
Avvio partita
Turno completo
Vittoria
Restart

Scenari minimi da automatizzare:

Scenario	Risultato atteso
Danno venti, scudo dieci	Scudo zero, salute meno dieci
Danno inferiore allo scudo	Salute invariata
Heal oltre massimo	Salute uguale a massimo
Cooldown zero	Non diventa negativo
Root attivo	Nessuna azione Move valida
Due unità stessa cella	Regola di conflitto applicata
Blast simultaneo letale	Entrambe completano il Blast
Ultima unità eliminata	Vittoria dopo Cleanup
Round-trip griglia	Cella iniziale recuperata
Timer scaduto	Lock-in automatico

Configurazione packaging

In **Project Settings** → **Maps & Modes**:

```
Editor Startup Map = L_Arena  
Game Default Map   = L_Arena  
Default GameMode   = BP_RTGameMode
```

In **Project Settings** → **Packaging**:

- configura le mappe da includere;
- usa Development durante il tutorial;
- abilita Shipping soltanto per la build finale;
- archivia simboli e log separatamente;
- verifica sempre la build su una cartella pulita.

Il cooking converte asset, Blueprint, audio e geometrie in formati runtime compatibili con la piattaforma; lo staging copia codice e contenuti cotti in una directory temporanea prima del package finale. ²⁵

Script PowerShell per i test

Scripts/Test.ps1:

```
$ErrorActionPreference = "Stop"

$UE = "C:\Program Files\Epic Games\UE_5.8"
$Project = "D:\Dev\ReactorTactics\ReactorTactics.uproject"
$Reports = "D:\Builds\ReactorTactics\TestReports"

& "$UE\Engine\Binaries\Win64\UnrealEditor-Cmd.exe" `
    $Project `
    -unattended `
    -nop4 `
    -nullrhi `
    -stdout `
    -FullStdOutLogOutput `
    -ExecCmds="Automation RunTest ReactorTactics;Quit" `
    -ReportExportPath=$Reports `
    -utf8output

if ($LASTEXITCODE -ne 0)
{
    throw "Unreal tests failed with exit code $LASTEXITCODE"
}
```

Script PowerShell per la build

Scripts/Build-Win64.ps1:

```
$ErrorActionPreference = "Stop"

$UE = "C:\Program Files\Epic Games\UE_5.8"
$Project = "D:\Dev\ReactorTactics\ReactorTactics.uproject"
$Archive = "D:\Builds\ReactorTactics\Win64"

& "$UE\Engine\Build\BatchFiles\RunUAT.bat" `
    BuildCookRun `
    -project=$Project `
    -noP4 `
    -platform=Win64 `
    -clientconfig=Development `
    -build `
    -cook `
    -allmaps `
    -stage `
    -pak `
    -archive `
    -archivedirectory=$Archive `
```

```

    -utf8output

if ($LASTEXITCODE -ne 0)
{
    throw "Packaging failed with exit code $LASTEXITCODE"
}

```

CI con GitHub Actions

Per Unreal è pratico usare un runner Windows self-hosted su una macchina dove siano già installati:

- Unreal Engine;
- Visual Studio Build Tools;
- Git LFS;
- SDK Windows;
- spazio disco sufficiente.

I runner self-hosted vengono installati e mantenuti dall'utente e possono essere etichettati per sistema operativo e capacità. ²⁶

Esempio `.github/workflows/unreal-ci.yml`:

```

name: Unreal CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test-and-package:
    runs-on: [self-hosted, Windows, X64, unreal]

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          lfs: true

      - name: Run tests
        shell: powershell
        run: ./Scripts/Test.ps1

      - name: Package Win64
        if: github.ref == 'refs/heads/main'
        shell: powershell
        run: ./Scripts/Build-Win64.ps1

```

Prima di usare il workflow in produzione:

- usa un repository privato;
- non eseguire pull request non fidate sul runner;
- limita i permessi del token;
- non conservare segreti nel workspace;
- pulisci periodicamente build e Derived Data;
- fissa versioni controllate delle action;
- archivia i log anche in caso di errore.

Uso futuro di C#

Dopo la vertical slice puoi riutilizzare C# creando script personalizzati per AutomationTool in:

```
Build/Scripts/
```

Epic documenta la creazione di progetti C# rilevati e caricati da AutomationTool per automatizzare processi specifici del progetto. ²⁷

Questo è il punto ideale per sfruttare l'esperienza C# senza rendere il gameplay dipendente da un runtime esterno.

Roadmap e timeline

Priorità post-tutorial

Le stime rappresentano settimane-persona full-time per uno sviluppatore che conosce ormai il progetto. Con lavoro serale o part-time possono diventare due-tre volte più lunghe.

Priorità	Funzionalità	Stima	Dipendenze
P0	Pulizia del resolver deterministico	1-2 settimane	Vertical slice
P0	Test completi dei conflitti	1-2 settimane	Resolver
P0	Server authoritative multiplayer	4-8 settimane	Resolver deterministico
P0	Replicazione delle azioni e GameState	2-4 settimane	Multiplayer base
P0	Timeout, disconnessione e reconnect	2-4 settimane	Networking
P0	Lobby privata	2-3 settimane	Online subsystem
P1	Quattro contro quattro	2-4 settimane	Performance e UI
P1	Sei eroi completi	8-16 settimane	Ability pipeline
P1	Draft o selezione personaggi	2-4 settimane	Roster
P1	Fog of war completo	3-6 settimane	LOS e networking
P1	Camouflage e last-known position	2-4 settimane	Fog of war

Priorità	Funzionalità	Stima	Dipendenze
P1	Power-up di mappa	1-3 settimane	Grid e status
P1	Replay deterministici	3-6 settimane	Event log e seed
P1	Spectator mode	3-5 settimane	Replay/networking
P1	Matchmaking	4-8 settimane	Backend online
P1	Ranking e stagioni	4-8 settimane	Matchmaking e database
P2	Ability mod/loadout	4-8 settimane	Roster stabile
P2	Migrazione selettiva a GAS	4-8 settimane	Abilità mature
P2	Tutorial interattivo	2-5 settimane	UX stabile
P2	Accessibilità	2-6 settimane	UI quasi definitiva
P2	Localizzazione	2-4 settimane	Testi centralizzati
P2	Controller	2-4 settimane	Navigazione UI
P2	Steam/Epic integration	3-6 settimane	Build stabile
P3	Console	8-20+ settimane	Certificazione e hardware
P3	Progressione cosmetica	6-12 settimane	Backend e contenuti
P3	Anti-cheat	Variabile	Multiplayer competitivo

Ordine consigliato per il multiplayer

Non iniziare dalla replica dei Pawn. Parti dai dati:

```

Player pianifica localmente
→ invia una richiesta al server
→ server valida
→ server registra l'azione
→ tutti i giocatori fanno lock
→ server risolve il turno
→ server replica gli eventi risultanti
→ client riproducono le animazioni

```

Il client non dovrebbe comunicare:

```

"Ho colpito il nemico e gli ho tolto trenta salute."

```

Dovrebbe comunicare:

```
"Voglio usare Ability.RifleShot  
dalla cella A  
verso la cella B."
```

Il server decide se il comando è valido e calcola il risultato.

Migrazione futura verso GAS

Valuta Gameplay Ability System quando esistono almeno:

- cinque-sei personaggi;
- decine di status;
- stacking rules;
- immunità;
- costi multipli;
- buff e debuff complessi;
- necessità di prediction online;
- molte interazioni tra effetti.

Non migrare tutto automaticamente. Griglia, resolver dei turni e conflitti di movimento possono rimanere sistemi autonomi; GAS può gestire attributi, costi, cooldown ed effetti.

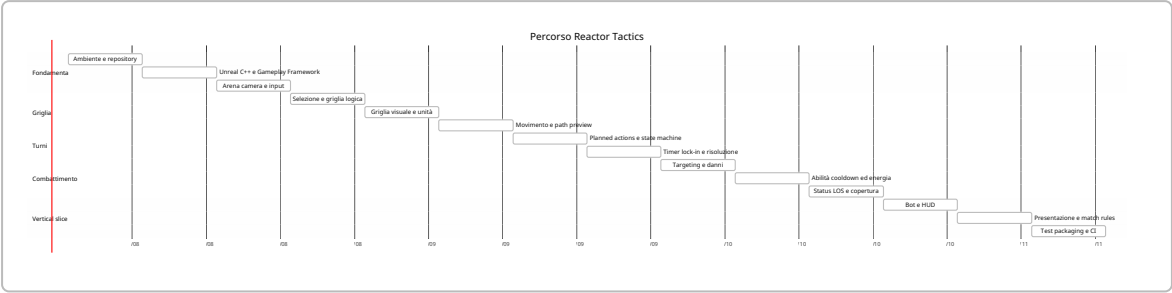
Criteri per una demo pubblica

Prima di pubblicare una demo servono:

Area	Requisito
Stabilità	Nessun crash noto nei flussi principali
Performance	Frame rate stabile sul PC minimo scelto
UX	Tutorial, tooltip e feedback di errore
Input	Rebinding o almeno mapping documentato
Audio	Volume configurabile
Display	Risoluzione e modalità finestra
Legalità	Licenze e attribuzioni verificate
Privacy	Nessuna telemetria non dichiarata
Save	Migrazione o reset gestito
Build	Firma e scansione antivirus
Supporto	Pagina con bug noti e contatti
Testing	Test automatici eseguiti in CI

Timeline indicativa

La timeline seguente assume inizio lunedì 3 agosto 2026, otto-dodici ore alla settimana e circa quattordici settimane.



Milestone

Milestone	Contenuto	Settimana prevista
Fondamenta	Toolchain, Git, classi base	Due
Sandbox	Camera, selezione, griglia	Cinque
Turn loop	Pianificazione e risoluzione	Otto
Combat loop	Abilità, danni e status	Undici
Vertical slice	Bot, HUD e vittoria	Tredici
Release interna	Test, packaging e CI	Quattordici

Il principio guida dell'intero percorso è semplice: **prima rendi corretta la decisione tattica, poi rendila bella da guardare**. Una capsula che si muove sulla cella giusta, risolve l'azione nella fase corretta e produce sempre lo stesso risultato è una base migliore di un personaggio spettacolare collegato a una logica fragile.

1

https://webcdn.triongames.com/atlasreactor/AtlasReactor_BetaGuide.pdf
https://webcdn.triongames.com/atlasreactor/AtlasReactor_BetaGuide.pdf

2

https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27

3

<https://www.unrealengine.com/news/unreal-engine-5-8-is-now-available>
https://www.unrealengine.com/news/unreal-engine-5-8-is-now-available

4

https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27

5

[Hardware and Software Specifications for Unreal Engine | Unreal Engine 5.8 Documentation | Epic Developer Community](https://dev.epicgames.com/documentation/en-us/unreal-engine/hardware-and-software-specifications-for-unreal-engine)
https://dev.epicgames.com/documentation/en-us/unreal-engine/hardware-and-software-specifications-for-unreal-engine

6

<https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install>
https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install

- 7 <https://github.com/microsoft/Winget-cli>
<https://github.com/microsoft/Winget-cli>
- 8 <https://learn.microsoft.com/th-th/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install>
<https://learn.microsoft.com/th-th/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install>
- 9 <https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-configure>
<https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-configure>
- 10 <https://docs.github.com/en/repositories/working-with-files/managing-large-files/installing-git-large-file-storage>
<https://docs.github.com/en/repositories/working-with-files/managing-large-files/installing-git-large-file-storage>
- 11 <https://github.com/github/gitignore/blob/main/UnrealEngine.gitignore>
<https://github.com/github/gitignore/blob/main/UnrealEngine.gitignore>
- 12 https://dev.epicgames.com/documentation/en-us/unreal-engine/get-started-with-ue4?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/get-started-with-ue4?application_version=4.27
- 13 <https://github.com/nxrighthere/UnrealCLR/blob/master/README.md>
<https://github.com/nxrighthere/UnrealCLR/blob/master/README.md>
- 14 <https://github.com/UnrealSharp/UnrealSharp>
<https://github.com/UnrealSharp/UnrealSharp>
- 15 <https://dev.epicgames.com/documentation/unreal-engine/unreal-automation-tool-overview-for-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/unreal-automation-tool-overview-for-unreal-engine>
- 16 https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?application_version=4.27
- 17 https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27
- 18 https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.3-release-notes?application_version=5.3
https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.3-release-notes?application_version=5.3
- 19 <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-user-interfaces-with-umg-and-slate-in-unreal-engine>
<https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-user-interfaces-with-umg-and-slate-in-unreal-engine>
- 20 <https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine>
- 21 25 <https://dev.epicgames.com/documentation/unreal-engine/packaging-your-project>
<https://dev.epicgames.com/documentation/unreal-engine/packaging-your-project>
- 22 <https://dev.epicgames.com/documentation/unreal-engine/run-automation-tests-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/run-automation-tests-in-unreal-engine>

- 23 https://dev.epicgames.com/documentation/en-us/unreal-engine/first-person-template?application_version=4.27
https://dev.epicgames.com/documentation/en-us/unreal-engine/first-person-template?application_version=4.27
- 24 <https://www.fab.com/listings/b2c95d5c-a805-460b-a01b-db6da3a778f0>
<https://www.fab.com/listings/b2c95d5c-a805-460b-a01b-db6da3a778f0>
- 26 <https://docs.github.com/actions/hosting-your-own-runners>
<https://docs.github.com/actions/hosting-your-own-runners>
- 27 https://dev.epicgames.com/documentation/unreal-engine/adding-automation-projects?application_version=4.27
https://dev.epicgames.com/documentation/unreal-engine/adding-automation-projects?application_version=4.27